



GITHUB.COM/AK-1612/NEXUS

Project Nexus : Enterprise LLM Token Optimization & Dynamic Model Routing

Architectural Patterns, Orchestration, and API Management

Strategic Optimization Overview



Business Challenge

High token consumption & unoptimized costs.



Proposed Solution

Dynamic routing via prompt complexity.



Core Deliverable

Azure API Management (APIM) infrastructure.



Research Scope

Cascades, speculative decoding, vector caching.

System Architecture & Orchestration



Central Orchestrator

Python execution engine for dynamic switching.



Data Validation

Strict schemas for standardized telemetry payloads.



Simulation Sandbox

Offline mock gateway routing tests.



Infrastructure as Code

Automated Azure Bicep provisioning.

Azure APIM Policy Implementation



Traffic Shaping:

Standard XML baseline access policies.



Enhanced Logic:

Dynamic, rule-based endpoint selection.



Gateway Governance:

Strict token limits and latency timeouts.



API Specifications:

Standardized OpenAPI endpoint contracts.

```
←!— azure-apim-policy-enhanced.xml →
<policies>
  <inbound>
    <base />
    ←!— Dynamic Routing Logic →
    <choose>
      <when
condition="@context.Variables["complexity"]
= "high")">
        <set-backend-service id="gpt-4o" />
      </when>
      <otherwise>
        <set-backend-service id="gpt-4o-mini"
/>
      </otherwise>
    </choose>

    ←!— Gateway Governance →
    <rate-limit-by-key calls="100"
renewal-period="60"

counter-key="@context.Subscription.Id)" />
  </inbound>
</policies>
```

Testing & Quality Assurance

01

Automated Suites

Comprehensive PyTest integration pipelines.

02

APIM Validation

Load-tested endpoint routing verification.

03

Component Tests

Orchestrator and schema stability validation.

04

Continuous Integration

GitHub Actions automated pipelines.

Copilot Ecosystem Integration

Internal Switching

GPT & Copilot environment routing.

User Experience

Transparent backend optimization with zero friction.

 .github/copilot-instructions.md

Copilot Integration Guidelines

Routing Rules

- Inline Completions: Always route to fast engine (gpt-4o-mini equivalent).
- Complex Refactoring: Prompt user to escalate to reasoning models.

Telemetry Standards

All generated telemetry payloads MUST adhere to
`schemas/telemetry\_schema.json`.

Fallback Mechanism

If the primary gateway is unreachable, automatically target the local sandbox at `localhost:8080/mock-gateway`.

Implementation Roadmap



Phase 1: Validation

- Finalize enhanced APIM policies and execute comprehensive PyTest suites across orchestrator endpoints.
- Validate edge cases for high-complexity prompt routing.



Phase 2: Benchmarking

- Evaluate orchestrator latency and token utilization performance against established 3rd-party startup routers.
- Compile performance metrics for executive review.



Phase 3: Enterprise Handoff

- Document comprehensive governance rules, standardize Bicep infrastructure template.
- Finalize repository deployment overviews for seamless enterprise adoption.